Know India — Project Report

Oct 24, 2026 · @PRATHAM SAHAI

1. Abstract and Overview

Know India is a full-stack web application that lets visitors explore India's 28 states and 8 union territories, their tourist places, festivals and the Constitution on one site. A React single-page app talks to an Express API backed by MySQL, signs users in with Google, and works offline as an installable Progressive Web App (PWA). This report describes how the code is built, what it does well, and where it falls short of its own documentation.

The analysis is based on a snapshot of the repository `DivyaAsthana8898/know-India-main` (commit `838a9bf`, 21 September 2026). The README credits the original author as `aryanjsx` and links to his published data packages; this report describes the code as it exists in the snapshot.

Aspect	Finding
Purpose	Interactive guide to Indian states, places, festivals and the Constitution
Size	107 files; about 19,900 lines of JavaScript, JSX and CSS
Frontend	React 18, React Router 7, Tailwind CSS, Three.js, Framer Motion; 21 routes plus a catch-all
Backend	Node.js 24 and Express 4; about 40 production endpoints
Storage	MySQL (TiDB Cloud compatible) plus static data from the `@aryanjsx/knowindia` npm package
Hosting	Frontend and backend deployed as separate Vercel projects
Automated tests	No test files; the CI workflow runs Jest with `--passWithNoTests`, a build and a backend syntax check

2. Features and User-Facing Modules

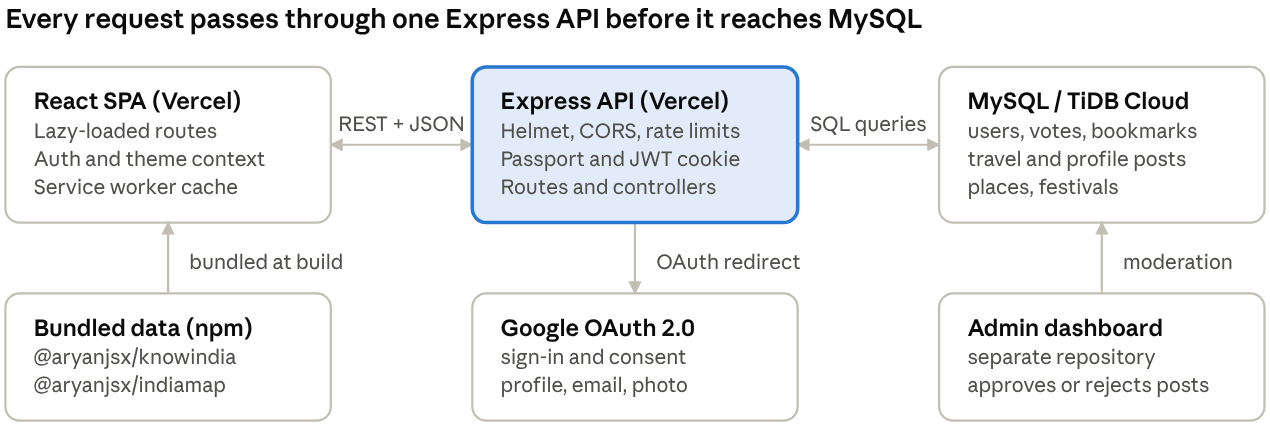
The site is organised around eight modules, each backed by its own page components and, where data is stored, its own API routes.

Module	What the user can do	Main files
India map	Click a state on an interactive map; a zoom animation leads to its page. A guided tour (react-joyride) explains the map	`IndiaMap.jsx`, `MapTour.jsx`
State and place pages	See a state's capital, languages and attractions; open a place with images and key facts	`StatePage.jsx`, `PlacePage.jsx`
3D district maps	View a state's districts as extruded 3D shapes; hover to highlight, click for the name	`State3DMap.jsx`
Festivals	Search festivals by name or month, 9 per page, and open a detail page	`FestivalsPage.jsx`, `FestivalDetailPage.jsx`
Constitution	Read the Preamble, overview, key features, amendments and the story of its drafting	`pages/constitution/` (5 pages)
Reviews and posts	Sign in, share a travel experience with photos, rate it, and up- or downvote others' posts	`ProfileAbout.jsx`, `Reviews.jsx`
Bookmarks	Save places; the list syncs to the account when signed in	`BookmarkButton.jsx`, `SavedPlaces.jsx`
Search and profile	One search box over states, attractions and festivals; edit the display name	`GlobalSearch.jsx`, `ProfileSettings.jsx`

Cross-cutting features include a persistent dark/light theme, SEO meta tags per page, an offline indicator, and a queue that stores feedback written offline and sends it when the connection returns.

3. System Architecture

The system has three tiers: a browser app, a stateless API, and a database. Static geography data is bundled into the app, so only user-generated and editable content needs the network.



system architecture · 3 tiers, 3 outside parties

The React app calls the API for places, festivals, posts, bookmarks and sign-in; the API is the only component that talks to MySQL. The separate admin dashboard shares the same database. The 3D maps also download district outlines from public GitHub files at runtime, outside this picture.

How a request flows

1. States, union territories and their attractions are read from the bundled `@aryanjsx/knowindia` package, with no network call.
2. Images and key facts for a state's places come from `GET /api/places/state/:name`, which queries MySQL through a pool of 10 connections.
3. Sign-in opens a popup to `/auth/google`; Passport verifies the user with Google, the API creates or updates a `users` row, signs a one-day HS256 token and sets it in an HttpOnly cookie.
4. Later calls send the cookie (`credentials: include`); the `authRequired` middleware verifies the token before a controller runs.

4. Frontend Design

The frontend keeps load time low by splitting code per route and by reading most data from a bundled package instead of the network.

Routing and loading. `App.js` wraps the router in a theme provider and an auth provider. The home page and the two auth result pages load immediately; 14 other pages and the 5 Constitution sub-pages are loaded with `React.lazy` behind a spinner. Pages that need a signed-in user (`/saved` , `/profile/*`) sit inside a `ProtectedRoute` wrapper.

State. There is no Redux. Two React contexts hold shared state: `AuthContext` (user, CSRF token, login and logout) and `ThemeContext` (dark or light, persisted). Context values are memoised with `useMemo` and `useCallback` to avoid needless re-renders.

Data adapter. `lib/knowIndia.js` wraps the npm data package and gives the rest of the app one set of helpers: URL slugs, lookup by slug or code, all places, and a text search. States are returned sorted by name and tagged as `state` or `union_territory` .

Search. `GlobalSearch.jsx` builds an in-memory index of states and attractions (festivals are added from the API) and scores each match: 100 for an exact name, 50 for a prefix, 30 for a substring, plus smaller points for word and keyword hits. Results are grouped and can be chosen with the keyboard.

3D district maps. `State3DMap.jsx` (708 lines) uses React Three Fiber and `OrbitControls` . It downloads a national outline and per-state district GeoJSON from public GitHub files, caches each download in module-level maps, applies 8 and 12 second time-outs, and thins every polygon ring to at most 120 points before extruding it. A name-matching table handles renamed states such as Orissa to Odisha and merges the two Dadra and Nagar Haveli, Daman and Diu files.

Styling and offline help. Tailwind CSS uses class-based dark mode and an extra `xs` breakpoint at 475 px. Feedback typed while offline is stored locally and retried five seconds after start-up and then every minute.

5. Backend and API Design

The backend is one Express app (`server.js` , 1,079 lines) plus five route files, three controllers and a few utilities. It is exported as a module for Vercel's serverless runtime and only calls `listen()` outside production.

Middleware order. Each request passes through, in order: Helmet (content security policy, frame denial, HSTS in production, referrer policy); a general rate limit of 200 requests per 15 minutes per IP; a CORS allow-list of four production domains plus a pattern for Vercel preview URLs; JSON and form parsers capped at 1 MB; the cookie parser; and Passport without sessions. Everything under `/auth` also passes a stricter limiter of 10 attempts per 15 minutes in production (50 in development).

Validation. Controllers check types and ranges before touching the database: IDs must be positive integers, ratings 1 to 5, names at most 100 characters, feedback at most 5,000 characters and image lists at most 10 items. User text is cleaned with `sanitize-html`, which strips every tag, and every SQL statement uses `?` placeholders, so the queries reviewed for this report are not open to SQL injection.

Group	Path	Access	Notes
Auth	`/auth/*`	Public and signed-in	Google login, callback, status, me, logout, CSRF token
Places	`/api/places/state/:name`, `/api/state/:name/place/:id`	Public	Joins places, categories and images
Festivals	`/api/festivals`, `/api/festivals/:id`	Public	`?search=` and `?month=` filters
Reviews	`/api/profile/posts`	Signed-in	9 handlers; new posts start as `pending`
Saved places	`/api/saved-places`	Signed-in	List, add, check, remove, clear
Profile	`/api/profile/settings`	Signed-in	Read and rename
Feedback	`/api/feedback`	Signed-in	Rating 1 to 5 plus text
Health	`/api/health`, `/api/db-test`	Public	Connection check

Debug and mock-feedback routes exist only when `NODE_ENV` is not `production`, which keeps server details out of the live site. A middleware also creates missing tables once per cold start, because serverless functions have no separate set-up step.

6. Database Design

The application creates six of its own tables at start-up or on first use and reads six more that are created elsewhere. All user-owned rows point to `users` and are deleted with the user (`ON DELETE CASCADE`).

Table	Key columns	Rules
`users`	`google_id`, `name`, `email`, `avatar`, `role`	`google_id` unique; role is `user` or `admin`
`travel_posts`	`user_id`, `content`, `rating`, `images` (JSON), `upvotes`, `downvotes`	Rating limited to 1 to 5 by a CHECK
`post_votes`	`post_id`, `user_id`, `vote_type`	One vote per user per post (unique pair)
`profile_posts`	`place_name`, `state`, `content`, `rating`, `images`, `status`	Status is `pending`, `approved` or `rejected`; default `pending`
`profile_post_votes`	`post_id`, `user_id`, `vote_type`	Unique pair, same as above
`saved_places`	`user_id`, `place_id`, `place_name`, `state_slug`, `image`	One bookmark per user per place
`places`, `categories`, `place_images`, `place_key_information`, `festivals`, `Feedback`	Read by the API	No `CREATE` statement in this repository; the README says the admin dashboard manages them

Moderation workflow. A review written through `/api/profile/posts` is stored with status `pending`. Only approved posts are meant to appear publicly, and approval happens in the separate dashboard against the same database. A migration in `db.js` adds the `status` column to older tables and sets missing values to `pending`.

Vote counts. Each post keeps `upvotes` and `downvotes` counters next to the per-user vote rows. The code reads the counters, adjusts them in application memory and writes them back, so two simultaneous votes on one post could overwrite each other. A single `UPDATE ... SET upvotes = upvotes + 1` inside a transaction would remove that risk.

Connections. `utils/db.js` creates one pool of up to 10 connections with a 20-second connect time-out, a 60-second idle time-out and keep-alive. In production it requires SSL and loads the ISRG Root X1 certificate from `backend/certs/` or from an environment variable.

7. Security and Authentication

The API applies layered defences against the common web attacks; the weak points are in how the login token is delivered and how CSRF and logout are enforced, which section 9 lists.

Layer	Control	Detail
Headers	Helmet	Content security policy limited to the site and named image and font hosts; frames denied; HSTS for one year with preload in production; MIME sniffing off
Identity	Google OAuth 2.0	Passport strategy with profile and email scopes; no server-side session
Token	JWT	HS256 fixed on both signing and verifying; one-day expiry; the server refuses to start in production without `JWT_SECRET`
Cookie	`auth_token`	HttpOnly, Secure in production, one-day lifetime
Abuse	Rate limits	200 requests per 15 minutes overall; 10 per 15 minutes on `/auth` in production, keyed by IP and User-Agent
Origins	CORS	Four named domains plus one preview-URL pattern; localhost only outside production
Input	Validation	Integer IDs, length caps, all HTML stripped, image URLs limited to http, https and image data URLs
Ownership	Authorisation	Update and delete compare the post owner with the user ID inside the token and answer 403 otherwise; the user ID is never taken from the request body
Errors	Hidden detail	Generic messages in production; debug routes disabled

The repository also ships a `SECURITY.md` with a production checklist and a disclosure address policy (report privately, reply within 48 hours). Several of its statements describe intent rather than the current code, as the next sections show.

8. PWA, Deployment and CI/CD

The site installs like an app and keeps working with a weak connection, but its automated checks cannot currently fail a build.

Offline support. `service-worker.js` (cache version 4) pre-caches five files on install: the offline page, the manifest and three icons. Other requests follow per-type rules, listed below. Old caches are deleted on activation, and the worker takes control immediately. The web manifest sets standalone display, a dark theme colour and the `en-IN` language.

Request	Strategy	If it fails
Page navigation and `index.html`	Network only, never cached, so a new deploy never shows a stale white page	Shows `offline.html`
`/api/places...`	Network first; successful answers are stored	Last stored answer, else a 503 JSON `Offline`
Images	Cache first, fetched once if missing	Nothing returned
JavaScript, CSS and other files	Network first; successful answers are stored	Stored copy if present

Deployment. The frontend and backend are separate Vercel projects. The frontend is a Create React App build with a rewrite of every path to `index.html` and no caching for the HTML file. The backend runs `server.js` through `@vercel/node`, declares Node 24, and its `vercel-build` script deletes the optional machine-learning packages (`faiss-node`, `onnxruntime`, `@huggingface`) so the function fits serverless limits. The embedding service then falls back to plain text matching.

CI. A GitHub Actions workflow runs on pushes and pull requests to `main` and `develop`. The frontend job installs with `--legacy-peer-deps`, runs Jest with `--passWithNoTests` and builds; the job and every step are marked `continue-on-error`, so none of them can turn the run red. The backend job runs only `node --check server.js`, which catches syntax errors but nothing else.

9. Code Review Findings and Limitations

Reading the code against its own README and `SECURITY.md` turned up 13 issues: 4 high, 5 medium and 4 low. Each was checked in the source; severity is this report's own judgement.

#	Severity	Finding	Where
1	High	Public review endpoints return every author's email address (`user_email`), including on the unauthenticated list	`profilePosts.controller.js` lines 178, 283; `posts.routes.js` lines 40, 73
2	High	`GET /api/profile/posts/:id` does not filter on `status` , so pending and rejected posts can be read by anyone who guesses an ID, bypassing moderation	`profilePosts.controller.js` `getPostById`
3	High	The login token is placed in the redirect URL (`/auth/success?token=`) and copied to `localStorage` , though `SECURITY.md` says tokens are never stored there; URLs leak through history, logs and referrers	`auth.routes.js` ; `AuthContext.jsx`
4	High	`csrfProtection` is written but never mounted, so the CSRF token the frontend sends is not checked; production cookies use `SameSite=None` (the document says `Lax`) and the API accepts form-encoded bodies, which skip the CORS preflight	`auth.middleware.js` ; `jwt.js` ; `server.js`
5	Medium	The logout blacklist is an in-memory `Map` ; each serverless instance has its own copy and loses it on restart, so a signed-out token may still work elsewhere for up to a day. Logout is also allowed as a `GET`	`jwt.js` ; `auth.routes.js`

#	Severity	Finding	Where
6	Medium	`/api/profile/posts/status-check` is public in production and returns the database name, post counts, recent posts and the raw error message on failure	`profilePosts.controller.js` `getPostStatusCounts`
7	Medium	The page lets users attach 5 images of up to 5 MB as base64 text, but the API rejects JSON bodies over 1 MB; accepted images are stored inside a JSON column and the public list returns all approved posts at once, with no paging	`ProfileAbout.jsx`; `server.js`; `profilePosts.controller.js`
8	Medium	`/api/places/city/:cityName` is declared after `/api/places/:stateName/:cityName`, which also matches it, so the city route is unreachable	`server.js` lines 786 and 935
9	Medium	Without a certificate file or variable, the production database connection falls back to `rejectUnauthorized: false`, which accepts any server certificate	`utils/db.js`
10	Low	The map page requests `/api/states`, which the backend does not define; the error is only logged and a built-in list is used	`IndiaMap.jsx`; `config.js`
11	Low	`/api/places/state/:name` runs one extra query per place (N+1), and every `connectToDatabase()` call first runs `SELECT 1`	`server.js`; `utils/db.js`
12	Low	There are no test files, and no CI step can fail the build; the README says CI runs ESLint, but the workflow has no lint step	`ci.yml`; `frontend/src`

#	Severity	Finding	Where
13	Low	Documentation drift: no `LICENSE` file despite the MIT badge; README lists `config/multer.js` and upload protections that do not exist; Node is 22+, 18+ and 24.x in three places; `AUDIT_REPORT.md` (January 2026) is partly out of date	`README.md`; `CONTRIBUTING.md`; `package.json`

Unused code adds noise: `authOptional`, `csrfProtection` and `sanitizeContent` are never called, and the vector-search service is initialised in development but no route queries it. The front-end API address is also fixed to the production backend, so local development needs a code edit.

10. Recommendations and Conclusion

Fixing the four high-severity items first closes the real privacy and login risks at low cost; the rest improves reliability and maintainability.

Do first (security)

1. Remove `user_email` from every public response, and make `getPostById` return only approved posts unless the requester owns the post.
2. Stop sending the token in the redirect URL and drop the `localStorage` fallback; rely on the HttpOnly cookie and one `postMessage` signal to the opener.
3. Mount `csrfProtection` on all `POST`, `PUT` and `DELETE` routes, remove the form-encoded body parser from the API, and make logout `POST` only.
4. Delete or protect `status-check`, and make the database connection fail closed when no certificate is found.

Do next (reliability)

1. Move the city route above the two-segment route, and either add `/api/states` or remove the call.
2. Replace the per-place query with one `IN` query, and use `pool.execute` directly instead of testing every connection.
3. Upload review photos to an image host (the content security policy already allows Cloudinary) and store only URLs; add paging to the review list.

4. Keep logged-out tokens in a shared store such as Redis, or shorten token life.

Then (quality)

1. Add tests for the auth middleware, the sanitising helpers and the search scoring, and let CI fail on lint, test and build errors.
2. Add the missing `LICENSE`, agree on one Node version, read the API address from `REACT_APP_API_URL` as `SECURITY.md` already says, and update the README and the audit file.
3. Build the one unchecked roadmap item, multi-language support.

Conclusion. Know India combines a bundled geography dataset, a lazy-loaded React app, 3D district maps and a small, parameterised Express API into a usable, installable guide to India. Its security design is thoughtful on paper, and most controls exist in code, but a few exposures (author emails, unmoderated post reads, URL tokens) and an inactive CI pipeline separate it from production quality. With the fixes above it would be a solid base for further features.

Sources

- DivyaAsthana8898/know-India-main: all code, README, `SECURITY.md`, `CONTRIBUTING.md` and `backend/AUDIT_REPORT.md`, snapshot of 21 September 2026 (commit `838a9bf`).